STANDALONE ARCHITECTURE AND SYSTEM-DESIGN REVIEW

Full System Architecture and Risk Review

From Google OAuth and permanent media ingestion through hybrid search, carousel generation, storage lifecycle, consistency, and disaster recovery. Issues are ranked by maximum system-performance impact and classified as primary bottlenecks or child issues.

77.107 GiB	12	26	21
free after leftover cleanup	primary performance bottlenecks	child issues of those bottlenecks	video files remaining (keep set)

CRITICAL WARNING — LEFTOVER DOWNLOADS MUST AUTO-CLEAN

Drive ERROR/no-Media caches, processed Drive caches after Media exists, and failed partials filled the volume and caused ENOSPC. Manual cleanup is not enough. The indexer must unlink leftovers in the success and ERROR paths. Keep upload, YouTube, active PROCESSING, and unknown/orphan files. This is a required control of bottleneck #1, not a child footnote.

Evidence boundary. Production facts are from the 14 August 2026 07:16 UTC read-only re-check plus the prior audit. No deploy, settings change, cache delete, or DB mutation was performed for this report. Auto-clean is required and is not yet implemented in the indexer paths.

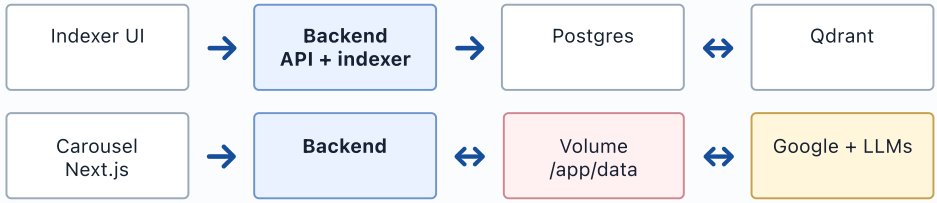
Review date	14 August 2026
System	Video/Image Search Indexer + Carousel Studio
Generated from	docs/full-system-architecture-and-risks.md
Rendering	Offline local Chromium · A4 · tagged PDF

Visual system atlas

Twelve views connect service topology, trust, data flow, consistency, retention, recovery, and the performance bottleneck → child map. Blue nodes are application-controlled; amber nodes are external or capacity-sensitive; red nodes are explicit failure boundaries.

1 · Complete topology

current production-confirmed service shape



2 · Trust boundaries

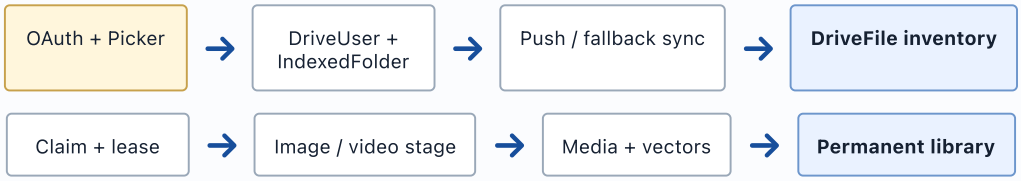
credentials, content, and derived intelligence

User/browser	Application trust	Data trust	Provider trust
queries · previews · Picker token · edits	**frontends · API · workers · policy enforcement**	Postgres tokens/metadata · Qdrant vectors · media bytes	Google · Gemini · Claude · OpenRouter · YouTube

Every crossing needs identity, least privilege, provenance, retention, and audit controls.

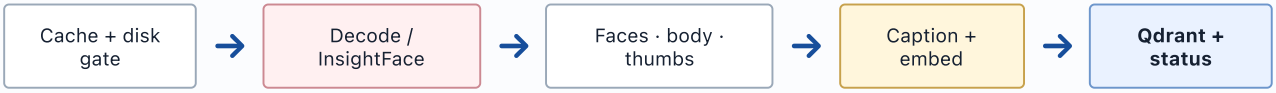
3 · Google Drive sync and index flow

OAuth through permanent library



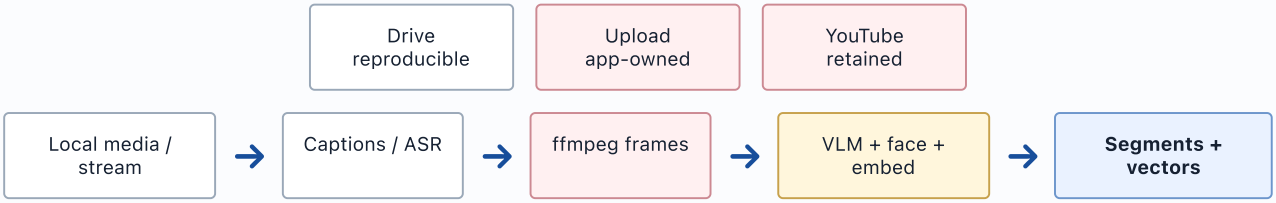
4 · Image pipeline

durable bytes to multimodal retrieval



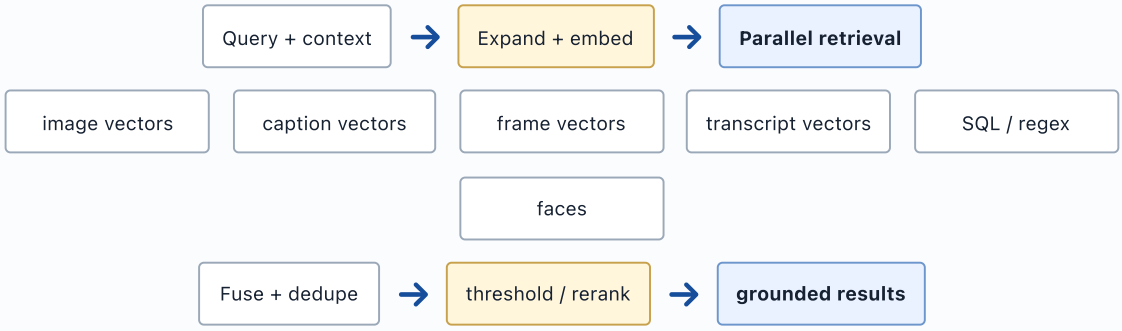
5 · Video pipeline

Drive, uploads, and retained YouTube converge



6 · Search and RAG

hybrid retrieval and evidence-aware ranking



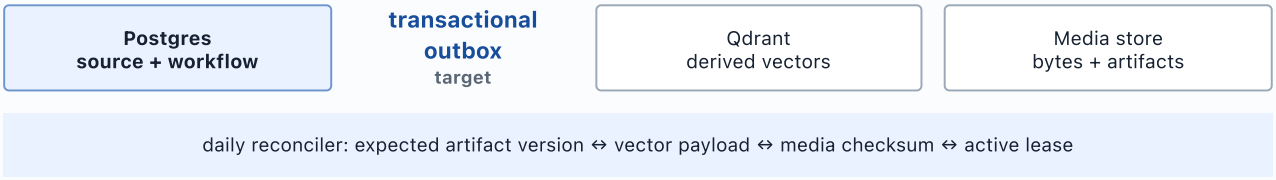
7 · Carousel and run-wide LLM flow

one immutable provider/model route per run

provider + model + transcript hash + prompt/algorithm version → generation provenance/cache key

8 · Data consistency flow

Postgres authority with derived asynchronous indexes



daily reconciler: expected artifact version ↔ vector payload ↔ media checksum ↔ active lease

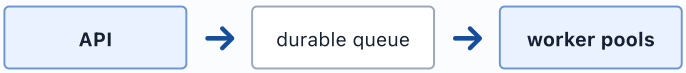
9 · Current versus target execution

separate failure domains and durable work

Current

shared CPU, DB, disk, process lifetime

Target



leases, fencing, independent scaling and restart

10 · Media retention lifecycle

source-aware policy, never filename-only deletion

KEEP upload / YouTube

AUTO-UNLINK Drive processed + Media

AUTO-UNLINK Drive ERROR / no-Media

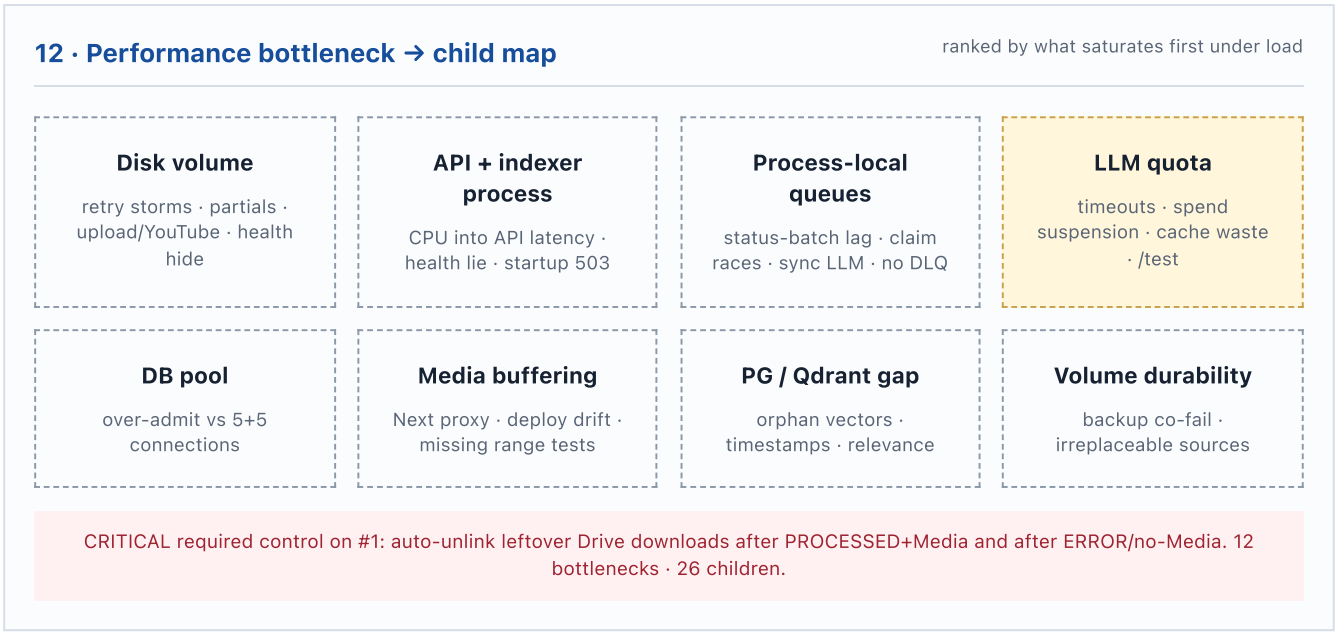
KEEP PROCESSING / orphan

CRITICAL: leftover Drive downloads must auto-clean after index. Manual cleanup is not a control. These leftovers filled the volume and caused ENOSPC.

11 · Failure containment and recovery

break amplification before replay

ENOSPC/auth/quota are not generic transient retries. Preserve source relations and isolate unaffected reads.



Full System Architecture and Risk Review

System: Video/Image Search Indexer, Permanent Media Library, and Carousel Studio **Review date:** 14 August 2026 **Evidence basis:** current local working tree plus a read-only production re-check at 14 August 2026 07:16 UTC (health, index, settings, volume, `/app/data/videos`, proof video). No production mutation, deployment, settings change, cache delete, or DB write was performed for this report. **Deployment boundary:** local code is not assumed deployed except where the re-check independently confirms a live fact.

1. Executive view

1.1 Business and system goals

The system turns long-lived media from Google Drive, local uploads, and retained YouTube sources into a searchable visual and transcript library. It supports five product outcomes:

1. connect a Drive account and choose an indexing root without destroying prior library work;
2. ingest images and videos into searchable metadata, face, caption, transcript, frame, and vector representations;
3. search across people, actions, scenes, spoken text, and folder context;
4. convert grounded video moments into Instagram carousel themes, topics, hooks, copy, and image selections; and
5. keep processed assets reproducible and reviewable while controlling storage, provider quota, and operational risk.

The immediate production constraint is storage and failure-domain coupling. The original incident filled the volume with leftover source downloads: **83.135 GiB across 1,353 cached video files**, **4,799 errors**, and sampled **ENOSPC**. Manual cleanups then removed **127 case-variant duplicates**, **182 processed Drive caches**, and **1,023 Drive ERROR/no-Media leftovers (41.391 GiB)**. The 14 August 2026 07:16 UTC read-only re-check shows those deletions still holding: **`/app/data` 77.107 GiB free (15.6% used)**, **21 files / 4.617 GiB remaining** in `/app/data/videos`, **0 leftover processed-Drive caches**, **0 leftover ERROR/no-Media caches**, **0 partials**. API `/health` is **200** and `/health/detail` reports DB, Drive, and Qdrant ok. Auto-index is **enabled**; `reindex_errored_files` and `reindex_skipped_files` remain **false**. Health is still a process-liveness check, not a disk-readiness gate.

CRITICAL WARNING — leftover source downloads must AUTO-CLEAN

> **CRITICAL:** leftover Drive source downloads (ERROR/no-Media, processed Drive caches after Media exists, and failed partials) filled the volume and caused **ENOSPC**. Manual cleanup is not enough. The indexer must **unlink leftovers in the success and ERROR paths** so this cannot recur. This is a **required control of bottleneck #1** (unbounded volume), not a child footnote of #13.

- **After successful index (Media committed):** the Drive source cache is leftover and must auto-delete. Drive remains the source of truth; Range streaming exists. Search, preview, frames, and carousel do not need the local Drive download once Media is committed.
- **After failed index with no Media:** the leftover download is leftover and must auto-delete. A retry re-downloads from Drive.
- **KEEP:** upload (the only copy), YouTube (re-download is unreliable), active **PROCESSING**, and unknown/orphan files until classified.
- Do not wait for another operator pass. If the success and ERROR paths leave bytes on `/app/data/videos`, the next index burst will refill the volume.

Production snapshot (read-only, 14 August 2026 07:16 UTC)

Check	Live value
Volume `/app/data`	91.361 GiB total · 14.238 used · **77.107 GiB free** (15.6% used)
`/app/data/videos`	**21 files · 4.617 GiB** · 0 partials · 0 leftover processed-Drive · 0 leftover ERROR/no-Media
Index counts	processed 1,656 · error 4,798 · processing 1 · skipped 3,088 API / 3,503 DB · archived 3 · pending 0
Media / Qdrant	Media 1,698 · video frames 11,384 · images 728 · captions 1,461
Settings	`auto_index_enabled=true` · `reindex_errored_files=false` · `reindex_skipped_files=false` · interval 30s
Proof `11bY89L94ctjD-mH3lo6Yb-fzLRfK3dDs`	**PROCESSING** · no Media · 300 frames · 24.78 MiB cache present · `error_message` null · claimed 06:42:01 UTC
Health	`/health` 200 · `/health/detail` DB/Drive/Qdrant ok · indexer `is_running=true` on the proof video
Prior cleanups still holding	127 case-variant dups · 182 processed Drive caches · 1,023 ERROR/no-Media

Check	Live value
Remaining keep set	YouTube 7 (3.671 GiB) · SKIPPED 11 · PROCESSING 1 · orphan/unknown 2

1.2 Evidence and confidence labels

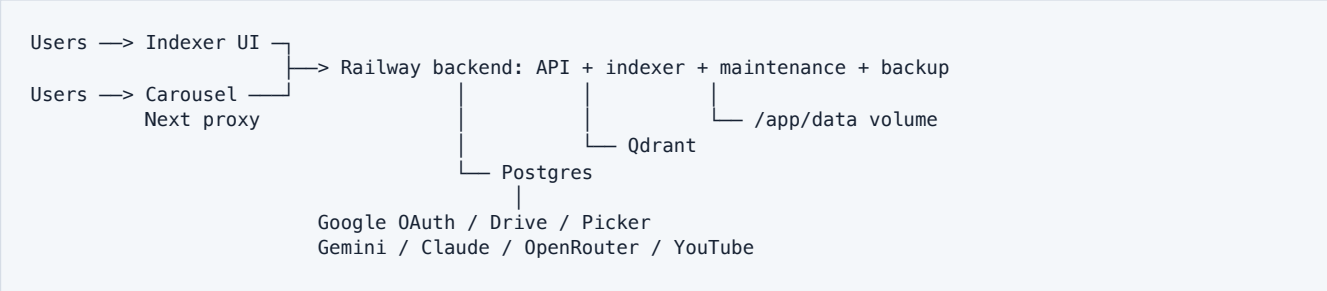
- **PROD-CONFIRMED:** observed in the 14 August 2026 07:16 UTC read-only re-check and the prior production audit: current free space and remaining keep-set files; prior leftover counts (127 / 182 / 1,023); ENOSPC samples; API health 200 during blocked indexing; current auto-index on and errored/skipped requeue off.
- **CODE-CONFIRMED:** directly visible in the current working tree.
- **LOCAL-ONLY:** implemented in the uncommitted working tree but not proven deployed.
- **ARCHITECTURAL RISK:** a credible failure mode inferred from design; not a claimed incident.
- **GOVERNANCE RISK:** privacy, retention, or control concern; not a legal conclusion.

Section 5 ranks every issue by **maximum system-performance impact** (throughput, latency, availability under load, disk, CPU, DB, LLM, queues), not by security or governance severity. Each item is a **Bottleneck** (the constraint that saturates first) or a **Child issue** (consequence, amplifier, or secondary defect of a parent bottleneck).

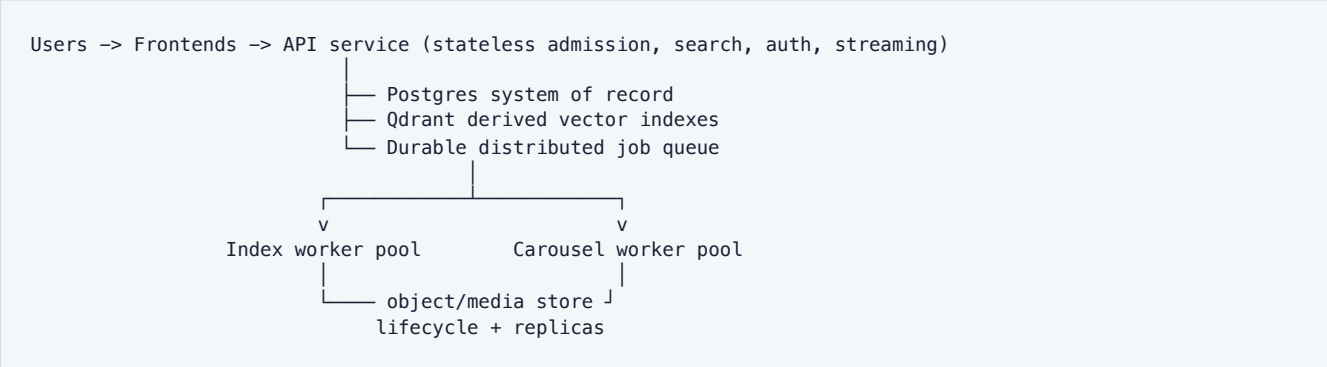
1.3 Deployed and external context

Production-confirmed Railway topology consists of a combined backend/API+indexer service, an Indexer UI, a Carousel Next.js service, a Drive connector/direct-client boundary, Postgres, Qdrant, and a persistent `/app/data` volume. External providers include Google OAuth, Drive API, Picker API, Gemini embeddings/VLM, optional Anthropic Claude, OpenRouter-routed models, and YouTube/yt-dlp. The working tree documents an optional same-image `dfi-indexer` split, but there is no evidence in this task that the split is deployed.

1.4 Current topology



1.5 Target topology



Target properties are independent API/indexer scaling, persisted jobs with leases and idempotency, object storage or an explicitly shared media tier, reconciliation between Postgres and Qdrant, bounded provider budgets, and tested backup/restore. A one-replica indexer is the safe first split because the local status batcher and process-local queue assume a single owner.

1.6 Trust boundaries and data ownership

Boundary	Sensitive material crossing	Current owner	Required control
Browser ↔ Indexer/Carousel	queries, folder choice, previews, generated copy	Frontends/backend	user authentication, CSRF/session policy, route authorization
Browser ↔ Google Picker	Drive access token, browser API key, selected folder	Google + browser	strict referrers, OAuth state integrity, least scopes
Backend ↔ Google	refresh/access tokens, Drive bytes, profile email	Postgres/backend	encryption, token rotation/revocation, audit trail
Backend ↔ LLM providers	captions, transcript excerpts, images/frames, prompts	provider APIs	minimization, routing policy, cost and retention controls
Backend ↔ Postgres	source identity, statuses, tokens, derived metadata	Postgres	authoritative transactions, backups, tenancy fields
Backend ↔ Qdrant	vectors and payload identifiers	Qdrant	derived-index versioning and reconciliation
Services ↔ volume	uploads, YouTube files, Drive cache, frames, refs, backups	persistent volume	category retention, quotas, manifests, restore plan

OAuth tokens are stored in `drive_users` and are the most sensitive credentials persisted by the application. Postgres is authoritative for source identity, status, Media relationships, transcript rows, carousel saves, and runtime settings. Qdrant is a rebuildable but operationally necessary derived index. Drive is authoritative for remote Drive source bytes; the application becomes authoritative for local uploads, retained YouTube copies, generated references, and any source no longer reproducible externally.

2. Complete topology and responsibilities

2.1 Railway services

- **Backend/API + indexer (production-confirmed combined):** FastAPI routes, deferred schema boot, OAuth, Drive listing and preview, search, carousel orchestration, auto-index loop, maintenance, and backups (`backend/app/main.py`).
- **Indexer UI:** account/folder selection, library/status, search, and operator controls (`frontend/src/app`).
- **Carousel:** main `/carousel` studio plus a substantial `/test` surface, using same-origin route handlers and `carousel-frontend/lib/api-proxy.ts`.
- **Optional indexer target:** same backend image with `RUN_INDEXER=true`, one worker/replica; API uses `RUN_INDEXER=false` (`docs/indexer-service.md`).
- **Data services:** Postgres and Qdrant; exact Railway plans, replicas, and region placement were not supplied.
- **Persistent volume:** `/app/data` categories include videos, durable media cache, thumbnails, frames, temporary/partial files, references, and backups.

2.2 Internal backend modules

Area	Primary modules	Responsibility
lifecycle	`main.py`, `db/schema.py`, `runtime_settings.py`	startup schema, settings, leader election, health
Drive	`drive/google_client.py`, `routers/drive_oauth.py`, `routers/drive.py`	OAuth, folder traversal, tokens, sync, stream/download
ingestion	`workers/indexer.py`, `pipelines/image.py`, `pipelines/video.py`	claims, media decode, face/frame/transcript extraction
storage	`drive/media_cache.py`, `storage.py`, `video/youtube_cache.py`	durable paths, free-space guard, atomic publication
vectors/search	`qdrant/*`, `search/moments.py`, `matching/service.py`	vector upserts, fusion, ranking, face matching
carousel	`routers/carousel_script.py`, `search/carousel_pipeline.py`, `search/carousel_frame_select.py`	themes through final image selection
LLM	`llm/carousel_llm.py`, `llm/openrouter.py`, `gemini/*`	run routing, model resolution, embeddings, VLM/rerank
operations	`workers/maintenance.py`, `workers/backup.py`, `scripts/cache_audit_cleanup.py`	backfill, recovery, backup, cache audit

3. End-to-end flows

3.1 Google OAuth and folder selection

1. Browser requests `GET /auth/google` with an optional `return_to`.
2. Backend allowlists the return origin and sends an Authorization Code request for `drive.readonly`, OpenID, email, and profile.
3. Google redirects to `/auth/google/callback`; backend exchanges the code without PKCE, fetches profile, and upserts one `DriveUser`.
4. Access/refresh tokens and expiry are stored in Postgres.
5. Browser calls `/api/drive-token`; backend refreshes near expiry and returns an access token, Picker API key, and project app ID to the browser.
6. Picker returns a folder; `/api/save-folder` verifies/resolves shortcuts, updates the selected folder, appends `IndexedFolder` history, and conditionally schedules sync.
7. Re-selecting a known folder reuses existing indexed data rather than wiping it.

Control observations: `return_to` has an origin allowlist in local code. OAuth `state` carries a URL rather than a server-bound nonce and the flow explicitly has no PKCE. `DriveUser` queries use `LIMIT 1`, so the active connection is effectively shared/global rather than request-tenant scoped.

3.2 Drive sync and permanent library

1. Leader startup seeds an in-memory file-list cache, persists listing state, and attempts Drive push-channel registration.
2. Push notifications trigger refresh; a six-hour default fallback full sync remains available.
3. The direct client breadth-first traverses folders in bounded waves, paginating 200 children and capping at 100,000 files.
4. `DriveFile` rows are upserted with source identity, path, size, content hash, folder root, and status.
5. Local permanence logic treats indexed history as additive and uses `ARCHIVED`/detach semantics without implying deletion of vectors or media.
6. Auto-index, if enabled in runtime settings, claims pending work and sends it to image/video paths.

Ownership: Drive listing is a remote observation; `DriveFile` is the internal inventory. The current local design intentionally preserves processed assets when a folder changes or a source disappears.

3.3 Image pipeline

```
Drive/upload row -> atomic claim -> durable cache/free-space gate -> decode
 -> InsightFace detection -> face/body metadata + thumbnails in Postgres/volume
 -> image/caption embedding -> Qdrant -> batched final status in Postgres
 -> maintenance caption/embed/re-id backfills
```

The worker uses bounded task sets, a DB semaphore, durable cache helpers, CPU offload, and an in-process `ImageEmbedQueue`. Embedding failures can still finalize a file as PROCESSED and rely on maintenance backfill, creating a deliberate temporary consistency gap. InsightFace defaults to CPU and 512-dimensional ArcFace vectors remain in Postgres; image and caption vectors are in Qdrant.

3.4 Video pipelines

Drive: claim row; ensure local video/cache as required; extract captions or ASR; sample frames with ffmpeg; run optional VLM enrichment and face detection; store `VideoSegment`, frame/face assets, and Qdrant frame/transcript vectors; finalize status.

Local upload: `/search/carousel/upload` writes up to 2 GiB to a temporary volume path, atomically publishes under an `upload:` ID, creates a `DriveFile(source="upload")`, then schedules priority indexing. This source is not reproducible from Drive and must be retained/backed up.

YouTube: `/youtube/videos` registers URLs/IDs. Background work can download a local retained copy, ingest captions, and create a `DriveFile(source="youtube")`. The audit confirms **3.671 GiB retained**. Cookies and upstream availability are runtime dependencies, but retained local bytes are the stable product source.

Timestamp chain: captions/VTT/ASR and sampled frames create segment start/end times. Qdrant transcript point IDs are based on file ID and start time. Search and carousel later use those timestamps for frame URLs and grounding, so timebase

drift is a correctness risk.

3.5 Range-aware playback and Carousel proxy

The production audit and old report established full-body buffering in the then-observed path. The current local tree is materially different and must not be described as deployed:

- `DriveDirectClient.stream_file_content` accepts `Range` and passes it to Drive.
- `/files/{id}/preview` returns a `StreamingResponse`, preserving status, content range/length, ETag, and last-modified for uncached Drive objects.
- cached video uses `FileResponse`.
- `carousel-frontend/lib/api-proxy.ts` passes through GET bodies for `206`, video, and octet-stream responses, while non-media responses and request bodies still use `arrayBuffer`.

Required production proof is a bounded byte-range canary through browser → Next → FastAPI → Drive with `206`, coherent headers, seek/cancel behavior, and RSS/network measurements. The separate `/download` endpoint still calls `download_to_memory` for non-YouTube Drive downloads in local code.

3.6 Search, RAG, and ranking

1. Parse query, person, folder, source, and action intent.
2. Expand/normalize text and produce Gemini embedding vectors.
3. Search image vectors, image-caption vectors, video-frame vectors, transcript vectors, SQL/regex transcript rows, and face relationships.
4. Run branches concurrently, deduplicate by `(file, timestamp)`, apply score thresholds and action contradiction checks.
5. Optionally use Gemini caption filtering/reranking and folder context.
6. Return ranked images and moments with source identifiers, snippets, preview URLs, and timestamps.

This is hybrid retrieval rather than a single atomic index. Relevance depends on embedding model/version, Qdrant payload correctness, transcript timing, thresholds in `config.py`, runtime toggles in Postgres, and LLM reranker availability.

3.7 Carousel themes → topics/hooks → copy → images → finalize

```
video selection -> transcript readiness/prerun
 -> themes (cached by transcript hash + effective provider/model)
 -> per-theme extract -> topics/hooks and timed evidence
 -> one carousel per selected hook/topic -> transcript guard
 -> copy generation/regeneration -> frame candidates
 -> grouped image ranking + user edits -> select-images/finalize
 -> autosave, feedback, references, export
```

Postgres advisory locks and row lock tokens coordinate expensive stages across workers; process-local locks are fast paths only. Generated saves carry `kind`, `theme_key`, model, transcript hash, input hash, layout, copy and algorithm versions, and JSON payload. The transcript guard snaps invented slide text back to indexed transcript evidence before image selection.

3.8 Run-wide LLM routing and cache keys

The local `resolve_carousel_llm` builds an immutable per-request pack from explicit request values plus runtime/env defaults. Effective providers are Claude direct, OpenRouter, or Gemini. `carousel_llm_cache_id` includes provider and effective model; theme and extract lookups compare the exact cache identity and transcript hash. This reduces model-mismatch reuse, but cache identity does not visibly include every prompt/template, decoding, policy, locale, or ranking-version input. The target cache key is a versioned canonical hash of provider, model revision, prompt/template version, transcript/content hash, generation parameters, locale, algorithm version, and policy flags.

3.9 Maintenance, backfill, and recovery

- startup retries schema/settings initialization and recovers stuck PROCESSING/transaction errors;
- a Postgres advisory lock elects one background leader;
- maintenance backfills captions, embeddings, re-identification, and cleanup work;
- stale carousel locks and orphaned processing are reclaimed;
- backup loop writes Postgres/Qdrant and carousel/deep-dive archives to the configured backup directory;
- local cache tooling can audit categories, dry-run, quarantine, and clean;

- status and vector gaps depend on periodic reconciliation/backfill.

Recovery is useful but startup-coupled: failed schema work can make all non-health routes return 503, while liveness remains green.

3.10 Deployment and configuration lifecycle

Build/deploy definitions exist per Railway service. Environment settings load through Pydantic and `.env`; mutable product/runtime toggles load from the singleton `app_settings` row. Startup runs `ensure_schema`, then loads runtime settings, then elects a leader. The Carousel README documents both `railway up` and source redeploy paths. Without a single release provenance field, manual CLI builds can diverge from Git commits and from one another. Target lifecycle: immutable image digest, migration job before rollout, release record containing Git SHA/config schema, environment validation, canary, and rollback to a known digest.

4. Data architecture

4.1 Postgres entities and roles

- **Identity/config:** `DriveUser`, `IndexedFolder`, `AppSettings`, `FolderContext`, `IndexingFolderPause`.
- **Inventory/lifecycle:** `DriveFile`, `FileIndexConflict`, status/error/decode/cache/archive fields.
- **Media graph:** `Media`, `Face`, `FaceEmbedding`, `FaceCluster`, `Person`, `Recognition`, `BodySignature`, `FaceWebMatch`, `OcrPage`, `VideoSegment`.
- **Carousel:** `CarouselGenerationSave`, `CarouselItemFeedback`, `CarouselItemReference`, and row-level carousel lock fields on `DriveFile`.

Postgres should remain the authoritative catalog and workflow ledger. A status must not be interpreted as proof that all volume and Qdrant artifacts exist.

4.2 Qdrant collections and consistency boundaries

Configured collections include `dfi_video_frames`, `dfi_images`, `dfi_image_captions`, and `dfi_video_transcripts`. Writes occur outside the Postgres transaction. Point IDs derive from stable source IDs and, for transcript points, rounded timestamps. Qdrant is eventually consistent with Postgres; any “PROCESSED” decision therefore needs an artifact manifest/version or reconciliation exception.

4.3 Volume categories and retention

Category	Reproducibility	Policy
upload source video	app is source of truth	KEEP; back up and replicate
retained YouTube source	upstream may disappear/change	KEEP; 3.671 GiB confirmed
Drive processed cache	reproducible while access persists	evict only after production Range proof
Drive ERROR/no-Media	uncertain/possibly redundant	conditional after remote access, quiescence, manifest
thumbnails/frames	derived but expensive	lifecycle by parent/version; rebuildable
reference uploads	user-authored input	KEEP unless explicit deletion policy
temp/partial	transient	quarantine/delete after owner lease + TTL
backups	recovery material	separate failure domain; tested retention

4.4 Source-of-truth matrix

Datum	Source of truth	Derived copies	Consistency rule
Drive bytes	Google Drive	volume cache	ETag/version/hash when available
upload bytes	app media store	processing cache	never delete without backup/policy
YouTube retained bytes	app media store after capture	frames/transcript	retain independently of upstream
file identity/status	Postgres `DriveFile`	UI caches	transactional state machine
media relationships/timestamps	Postgres	Qdrant payload	reconcile by file/media/version
vectors	Qdrant/pgvector by class	none	rebuild from versioned source artifacts
OAuth tokens	Postgres	process memory/browser access token	tenant scope, expiry, revoke/rotate
runtime settings	Postgres singleton + env fallback	process cache	expose effective config revision
generated carousel	Postgres save payload	browser state/export	key by full generation provenance

4.5 Lifecycle and deletion rules

Deletion must be source-based, not filename-based. A reconciler joins source type, source availability, `DriveFile`/`Media`, vector version, active lease, cache path, size/hash, and backup status. Every deletion run writes a dry-run manifest, requires category policy approval, uses bounded batches, stops on mismatches, and emits a post-run manifest. `ARCHIVED` and Drive disconnect do not imply deletion.

5. Performance ranking and bottleneck classification

Severity in this section is **maximum system-performance impact**: how hard the issue hits throughput, latency, availability under load, and saturation of disk, CPU, database connections, LLM quota, and queues. Security, privacy, and maintainability items stay in the register but rank lower unless they admit unbounded work or collapse a live resource.

Bottleneck: a primary constraint that limits system performance or capacity — the resource or control plane that saturates first. **Child issue:** a consequence, amplifier, or secondary defect of a parent bottleneck. Example: a retry storm is a child of a full volume plus auto-requeue; status-batch visibility lag is a child of process-local queues.

Twelve items are primary bottlenecks. The other twenty-six are children. Worst performance impact is first.

5.1 Ordered performance register

Rank	Class	Issue	Saturates / amplifies	Parent bottleneck(s)	Perf band
1	**Bottleneck**	Full/unbounded persistent volume	Disk (ENOSPC)	—	Critical
2	**Bottleneck**	API/indexer colocation	Shared process CPU, RSS, event loop	—	Critical
3	**Bottleneck**	CPU/InsightFace/ffmpeg contention	CPU and memory bandwidth	— (amplified by #2)	Critical
4	**Bottleneck**	DB pool and concurrency saturation	Postgres connections	—	Critical
5	**Bottleneck**	No durable distributed job queue	Admission, leases, backpressure	—	Critical
6	**Bottleneck**	LLM quota contention	Gemini/Claude/OpenRouter quota	—	Critical
7	Child	Retry storms	Disk, Drive, DB, LLM, queues	#1 volume + #5 queue	High
8	**Bottleneck**	Drive preview/download buffering	Memory, egress, event loop	—	High
9	Child	Next proxy buffering/version drift	Carousel hop memory/timeouts	#8 media buffering	High
10	Child	Long synchronous LLM requests	Worker slots, proxy time, duplicate spend	#5 queue + #6 LLM	High
11	Child	Status batch visibility lag	Operator/recovery truth	#5 queue	High
12	Child	Video claim races and orphan ownership	Duplicate ffmpeg/Drive/LLM	#5 queue	High
13	Child	Partial-file and cache cleanup lifecycle	Disk headroom and unsafe deletes	#1 volume	High
14	**Bottleneck**	Drive listing scale	Memory, Drive quota, event loop	—	High
15	**Bottleneck**	Postgres/Qdrant non-transactional consistency	Searchable capacity and rework	—	High
16	Child	Stale/orphan vectors	Qdrant scan cost and wrong hits	#15 consistency	High
17	Child	Health semantics hide blocked indexing	Ops and rollout decisions	#1 volume + #2 colocation	High
18	Child	Schema migration/startup coupling	All non-health routes on boot	#2 colocation	High
19	**Bottleneck**	External provider outage/fallback	External dependency availability	—	Medium-high
20	Child	Cost/egress/model-spend controls	Provider suspension = capacity loss	#6 LLM + #7 retries	Medium-high
21	Child	Observability gaps	Delayed detection of every saturator	#1–#6, #8, #14, #15	Medium-high

Rank	Class	Issue	Saturates / amplifies	Parent bottleneck(s)	Perf band
22	Child	Capacity planning and SLO absence	Recurring saturation	#1–#6 (meta)	Medium-high
23	**Bottleneck**	Single-volume/single-region durability	Availability after volume/region loss	—	Medium-high
24	Child	Backup/restore and DR uncertainty	Recovery of metadata and media	#23 durability	Medium-high
25	Child	Upload/YouTube retention dependency	Irreplaceable bytes on the saturating volume	#1 volume + #23 durability	Medium-high
26	Child	LLM cache poisoning/model mismatch	Wasted quota and duplicate generation	#6 LLM	Medium-high
27	Child	Secrets/config/runtime-settings drift	Mis-tuned concurrency and routing	#2, #4, #6	Medium-high
28	**Bottleneck**	OAuth refresh/token lifecycle	Drive-path availability	—	Medium-high
29	Child	`railway up` versus Git deployment drift	Can ship the buffering/old media path	#8 + #9	Medium
30	Child	Transcript grounding/timestamp correctness	Wrong frames and rework	#15 consistency	Medium
31	Child	Public `/test` and admin-like surface	Unbounded costly admission if exposed	#6 LLM + #20 spend	Medium
32	Child	Shared Drive account / tenant isolation	One Drive identity and quota	#14 listing + #28 OAuth	Medium
33	Child	Data privacy/PII/face biometric governance	Not a live capacity constraint	— (governance)	Medium
34	Child	Google Picker referrer/config coupling	Onboarding, not throughput	#28 OAuth + #27 config	Medium
35	Child	Monolithic backend and Carousel files	Change risk to performance paths	#22 capacity model	Medium
36	Child	Duplicated main versus `/test` UI	Divergent controls on costly routes	#31 `/test`	Medium
37	Child	Weak frontend test coverage	Missed buffering/range regressions	#8 + #9	Medium
38	Child	Search relevance regression detection	Quality, not a saturating resource	#15 + #6 + #22	Medium

5.2 Parent → child groups

```
#1 DISK volume ── REQUIRED: auto-unlink leftover Drive downloads
                            ├─► #7 retry storms
                            ├─► #13 missing auto-clean implementation
                            ├─► #25 upload/YouTube compete for same mount
                            └─► #17 health hides blocked writes

#2 COLOCATION ──────────────► #17 health 200 while indexer blocked
                            ├─► #18 schema/startup 503 with green liveness
                            └─► #27 mis-tuned shared concurrency
                           (amplifies #3 CPU into API latency)

#3 CPU / InsightFace / ffmpeg  (primary compute saturator; no exclusive child)

#4 DB POOL ─────────────────► #27 config that over-admits vs pool size

#5 PROCESS-LOCAL QUEUE ─────► #11 status-batch visibility lag
                            ├─► #12 claim races / orphan rework
                            ├─► #10 long sync LLM (no persisted job)
                            └─► #7 retry storms (no DLQ / budget)

#6 LLM QUOTA ───────────────► #10 long sync LLM / proxy timeout
                            ├─► #20 spend → provider suspension
                            ├─► #26 cache mismatch wastes quota
                            └─► #31 /test unbounded generation if exposed

#8 MEDIA BUFFERING ─────────► #9 Next proxy buffering
                            ├─► #29 railway up can ship the old path
                            └─► #37 missing range/proxy tests

#14 DRIVE LISTING ──────────► #32 single DriveUser / shared quota

#15 PG / QDRANT CONSISTENCY ─► #16 stale/orphan vectors
                            ├─► #30 timestamp / grounding rework
                            └─► #38 relevance regressions

#19 PROVIDER OUTAGE ────────  (availability of OAuth, embed, carousel)

#23 VOLUME DURABILITY ──────► #24 backup/DR co-failure
                            └─► #25 irreplaceable upload/YouTube loss

#28 OAUTH LIFECYCLE ────────► #34 Picker/referrer break
                            └─► #32 shared token/tenant

Meta amplifiers: #21 observability and #22 SLO/capacity model sit under every primary bottleneck.
#33 privacy has no capacity parent. #35/#36 are change-risk children of #22/#31.
```

Parent bottleneck	Child issues
#1 Full/unbounded volume	**Required control: auto-unlink leftover Drive downloads after index.** Children: #7 retry storms; #13 missing auto-clean implementation; #17 health hide; #25 upload/YouTube on same mount
#2 API/indexer colocation	#17 health hide; #18 schema/startup coupling; #27 config drift; amplifies #3 into API latency
#3 CPU/InsightFace/ffmpeg	none exclusive; saturates workers and, with #2, the API event loop
#4 DB pool saturation	#27 config that admits more jobs than connections
#5 No durable job queue	#7 retry storms; #10 long sync LLM; #11 status-batch lag; #12 claim races
#6 LLM quota contention	#10 long sync LLM; #20 spend/suspension; #26 cache waste; #31 `/test` admission
#8 Drive preview/download buffering	#9 Next proxy; #29 deploy drift; #37 weak frontend tests
#14 Drive listing scale	#32 shared Drive identity/quota
#15 Postgres/Qdrant consistency	#16 orphan vectors; #30 timestamp correctness; #38 relevance detection
#19 Provider outage/fallback	no exclusive child; collapses embed/sync/carousel capacity

Parent bottleneck	Child issues
#23 Single-volume durability	#24 backup/DR uncertainty; #25 upload/YouTube retention
#28 OAuth token lifecycle	#32 shared tenant token; #34 Picker config

5.3 Ranked issue register

Evidence uses file paths for code facts and explicit labels for audit facts. Class and parent fields are the performance taxonomy; they do not replace the evidence labels.

---

1. Critical bottleneck — full/unbounded persistent volume

> **Class:** Bottleneck. **Saturates:** disk. **Children:** #7, #13, #17, #25. **Likelihood:** High. **Blast radius:** all ingestion, uploads, frames, backups, and any service sharing the mount. **Evidence:** PROD-CONFIRMED original fill 83.135 GiB/1,353 files and ENOSPC; leftover classes that caused it were Drive ERROR/no-Media, processed Drive caches after Media, case-variant dups, and failed partials. Manual reclaim held at re-check: **77.107 GiB free**, **21 keep-set files / 4.617 GiB**, 0 leftover processed-Drive, 0 leftover ERROR/no-Media. CODE-CONFIRMED the indexer still has no success/ERROR unlink of Drive source cache (`pipelines/video.py`, `storage.py`, `media_cache.py`, `cache_audit_cleanup.py` is operator-only). **Failure mode:** leftover downloads accumulate until writes fail; retries create more I/O; logical/physical state diverges. Throughput of the ingest plane falls to zero while the volume is full. **Required control:** auto-unlink Drive leftovers after Media commit and after ERROR with no Media. Manual cleanup is not a control. **Detection:** used/free bytes and inodes by category, leftover Drive-cache count after terminal status, write-rejection count, oldest partial, growth slope. **Short term:** implement indexer unlink on PROCESSED+Media and ERROR/no-Media; keep `reindex_errored_files` false until that ships; enforce 70/80/85% thresholds. **Target:** lifecycle-managed object/media tier with quotas, protected categories, high/low-water eviction, and replicas.

---

2. Critical bottleneck — API/indexer colocation

> **Class:** Bottleneck. **Saturates:** shared process CPU, RSS, event loop, DB slots, and disk I/O. **Children:** #17, #18, #27. Amplifies #3. **Likelihood:** High. **Blast radius:** all interactive APIs and background processing. **Evidence:** PROD-CONFIRMED API health 200 while indexing was blocked. CODE-CONFIRMED `backend/app/main.py` starts API, auto-index, maintenance, and backup; `docs/indexer-service.md` is optional. **Failure mode:** CPU, memory, DB, Drive slots, and disk contention degrade API while liveness stays green. **Detection:** per-role CPU/RSS/event-loop lag, API latency under index load, dependency readiness. **Short term:** use `RUN_INDEXER` role gate after reviewed deployment plan; cap worker concurrency. **Target:** independent API/indexer/carousel workers linked by durable queue and shared data contracts.

---

3. Critical bottleneck — CPU/InsightFace/ffmpeg contention

> **Class:** Bottleneck. **Saturates:** CPU and memory bandwidth during video bursts. **Amplified by:** #2. **Children:** none exclusive. **Likelihood:** High during video bursts. **Blast radius:** event loop, search latency, throughput. **Evidence:** CPUExecutionProvider and ffmpeg/frame settings in `config.py`; CPU helpers and worker semaphores. **Failure mode:** oversubscription and memory bandwidth saturation create nonlinear latency. After an API/indexer split this remains the worker-pool saturator. **Detection:** CPU steal/load, run queue, ffmpeg count, stage p95, event-loop lag. **Short term:** cap concurrent CPU stages and separate API process resources. **Target:** resource-class worker pools with cgroup limits, autoscaling, and benchmark-derived concurrency.

---

4. Critical bottleneck — DB pool and concurrency saturation

> **Class:** Bottleneck. **Saturates:** Postgres connections. **Children:** #27. **Likelihood:** High under load. **Blast radius:** status truth, OAuth, search, indexing. **Evidence:** `backend/app/config.py` records prior 46-job/pool≈24 timeout behavior and configures image 40, DB 24, pool 5+5 per worker. **Failure mode:** pool timeout causes false failures and retry amplification; admin connections are starved. **Detection:** checked-out/waiters/timeouts, transaction age, lock/deadlock rate, connections by service. **Short term:** set explicit per-service connection budgets and reduce admission concurrency. **Target:** pooler plus short transactions, queue-based backpressure, and load-tested budgets.

5. Critical bottleneck — no durable distributed job queue

> **Class:** Bottleneck. **Saturates:** admission, ownership, and backpressure. **Children:** #7, #10, #11, #12. **Likelihood:** High on restart/scale. **Blast radius:** image embeddings, status flushes, user-triggered generations. **Evidence:** `backend/app/workers/embed_queue.py` uses `asyncio.Queue`; `worker` tracks process tasks; status batching has single-owner guidance. **Failure mode:** queued work disappears on restart, duplicates across replicas, or remains PROCESSING until recovery. Without a queue, admission cannot protect disk, CPU, DB, or LLM. **Detection:** compare claimed rows with active leases; queue depth/age; orphan recoveries per restart. **Short term:** keep one indexer replica, persist claim timestamps/attempts, drain on shutdown. **Target:** Postgres-backed or managed queue with leases, visibility timeout, idempotency, DLQ, and replay.

6. Critical bottleneck — LLM quota contention

> **Class:** Bottleneck. **Saturates:** Gemini, Claude, and OpenRouter quota. **Children:** #10, #20, #26, #31. **Likelihood:** High. **Blast radius:** indexing captions, search rerank, carousel generation. **Evidence:** shared Gemini semaphore families and high configured parallelism in `config.py`; Claude/OpenRouter routes share product demand. **Failure mode:** background backfill consumes quota needed for interactive generation; rate-limit retries compound latency. **Detection:** requests/tokens/cost and 429s by provider/model/workload, queue wait, fallback use. **Short term:** reserve interactive quota and pause backfill on 429/latency thresholds. **Target:** provider gateway with workload budgets, priorities, spend caps, and model fallback policy.

7. High child — retry storms

> **Class:** Child issue. **Parents:** #1 volume (ENOSPC + auto-requeue) and #5 queue (no DLQ or retry budget). **Likelihood:** High during deterministic provider/capacity failure. **Blast radius:** Drive, disk, DB, Qdrant, LLM quotas. **Evidence:** PROD-CONFIRMED containment disabled auto-index/requeue; `auto_indexer.py`, `requeue_failed.py`, and transcript Qdrant upsert retries. **Failure mode:** unchanged errors repeatedly download/process and increase backlog/cost. This is the amplifier that turns a full volume into 4,799 errors. **Detection:** attempts by error class, retry rate, next-attempt age, repeated byte count. **Short term:** circuit-break ENOSPC/auth/quota classes; bounded exponential backoff with jitter. **Target:** centralized retry policy, budget, DLQ, operator replay, and dependency-aware admission.

8. High bottleneck — Drive preview and download buffering

> **Class:** Bottleneck. **Saturates:** memory, egress, and the event loop on the media path. **Children:** #9, #29, #37. **Likelihood:** Medium until production proof. **Blast radius:** memory, egress, seek experience, cache-eviction safety. **Evidence:** LOCAL-ONLY Range stream in `routers/drive.py` and `drive/google_client.py`; `/download` still calls `download_to_memory`. **Failure mode:** a route or deployment version buffers a full large file, causing RSS spikes/timeouts. **Detection:** bytes requested vs transferred, response status, RSS delta, cancellation latency. **Short term:** integration-test preview and download separately; block processed-cache eviction until canary passes. **Target:** one shared bounded-range/stream abstraction with HEAD, 206, 416, cancellation, and memory limits.

9. High child — Next proxy buffering/version drift

> **Class:** Child issue. **Parent:** #8 media buffering (same memory/timeout class on the Carousel hop). **Likelihood:** Medium. **Blast radius:** Carousel playback and long API calls. **Evidence:** LOCAL-ONLY media streaming branch in `carousel-frontend/lib/api-proxy.ts`; JSON and request bodies remain buffered; production version unverified. **Failure mode:** stale deployment buffers videos or strips headers; long responses exceed platform limits. **Detection:** deployed SHA, response header conformance, proxy RSS, upstream/downstream byte counts. **Short term:** add route tests and verify deployed artifact digest. **Target:** streaming reverse proxy with explicit body limits, timeout classes, and release provenance.

10. High child — long synchronous LLM request/proxy timeout

> **Class:** Child issue. **Parents:** #5 queue (no persisted async job) and #6 LLM (long provider calls). **Likelihood:** High for large transcripts. **Blast radius:** Carousel UX, duplicate spend, backend workers. **Evidence:** route handler comments around long upstream waits and many synchronous carousel endpoints. **Failure mode:** client/proxy disconnects while provider call continues; user retries duplicate work and hold worker slots. **Detection:** end-to-end stage latency, disconnect/cancel count, duplicate input hashes. **Short term:** idempotency keys and persisted status for long stages; explicit timeout classes. **Target:** asynchronous generation jobs, progress events, cancellation, resumable results.

11. High child — status batch visibility lag

> **Class:** Child issue. **Parent:** #5 process-local queues (`IndexStatusBatcher`). **Likelihood:** High by design. **Blast radius:** UI/operators and recovery decisions. **Evidence:** `INDEX_STATUS_BATCH_SIZE=100`, `IndexStatusBatcher`, and docs single-owner note. **Failure mode:** completed work appears PROCESSING; restart loses pending status writes and triggers adoption/rework. **Detection:** batch queue depth/oldest age, stage-complete vs status-final lag. **Short term:** time-based flush in addition to size; flush on shutdown. **Target:** durable event/outbox and materialized status projection.

12. High child — video claim races and orphan ownership

> **Class:** Child issue. **Parent:** #5 no durable queue lease or fencing token. **Likelihood:** Medium. **Blast radius:** duplicate ffmpeg/Drive/LLM work and status corruption. **Evidence:** advisory claim/execution locks and orphan adoption in `backend/app/workers/indexer.py`; stale thresholds in `config.py`. **Failure mode:** worker death or lease ambiguity causes duplicate adoption or long stuck PROCESSING rows. **Detection:** duplicate active source IDs, lock wait, claim age, recovery count. **Short term:** one indexer replica; expose owner token/heartbeat; require compare-and-set finalization. **Target:** durable queue lease and fencing token on every side effect.

13. High child — partial-file and cache cleanup lifecycle

> **Class:** Child issue. **Parent:** #1 unbounded volume. This is the missing implementation of #1's required auto-clean control, not a separate policy debate. **Likelihood:** High. **Blast radius:** disk headroom and cache correctness. **Evidence:** PROD-CONFIRMED leftovers that filled the volume were later removed only by operator jobs (127 case-variant dups, 182 processed Drive caches, 1,023 ERROR/no-Media). Re-check: those classes are now 0 on disk, but the indexer still does not unlink on success/ERROR. Temporary/partial/atomic paths remain in media/upload code. **Failure mode:** terminal index leaves Drive bytes behind; crashes leave partials; a later cleanup can delete active/KEEP files if it is filename-based. **Detection:** leftover Drive-cache count after PROCESSED+Media or ERROR/no-Media; partial age/owner; duplicate hash groups. **Short term:** unlink Drive leftovers in the indexer success and ERROR paths; quarantine partials by TTL only after lease checks; never delete upload/YouTube/PROCESSING/orphan in those paths. **Target:** object-store multipart lifecycle plus reconciler with ownership/fencing.

14. High bottleneck — Drive listing scale

> **Class:** Bottleneck. **Saturates:** process memory, Drive API quota, and event-loop time during sync. **Children:** #32. **Likelihood:** Medium-high. **Blast radius:** sync freshness and API/event-loop capacity. **Evidence:** breadth-first in-memory results, 100,000 cap, 200-page size, fallback full sync in `drive/google_client.py`. **Failure mode:** long scans, memory growth, API quota pressure, truncated library. **Detection:** folders/files/pages, duration, quota, truncation, change lag. **Short term:** surface truncation prominently and avoid unnecessary full scans. **Target:** Drive Changes API cursor, incremental checkpoints, partitioned inventory reconciliation.

15. High bottleneck — Postgres/Qdrant non-transactional consistency

> **Class:** Bottleneck. **Saturates:** effective searchable capacity and forces rework. **Children:** #16, #30, #38. **Likelihood:** High over time. **Blast radius:** missing/incorrect search and false PROCESSED status. **Evidence:** separate commits/upserts across `pipelines/*`, `qdrant/*`, and `embed_queue.py`; embed failure may still finalize PROCESSED. **Failure mode:** DB commits without vectors or vectors survive changed/deleted rows. "Processed" count overstates throughput. **Detection:** artifact completeness reconciliation by file/version; orphan vector scans. **Short term:** mark `vector_state`/version and backfill gaps; do not equate status with completeness. **Target:** transactional outbox, idempotent vector consumer, and reconciler.

16. High child — stale/orphan vectors

> **Class:** Child issue. **Parent:** #15 non-transactional Postgres/Qdrant writes. **Likelihood:** High with archive/reindex/model changes. **Blast radius:** search quality and privacy deletion guarantees. **Evidence:** archived rows explicitly retain vectors; point IDs encode source/timestamp but not model version; no complete delete path shown. **Failure mode:** old embeddings remain queryable or collide with regenerated timestamps. Extra Qdrant scan cost follows. **Detection:** Qdrant payload IDs absent from Postgres; version distribution; delete verification. **Short term:** filter on active/current version and run dry-run orphan reports. **Target:** versioned collections/aliases, tombstone consumer, and deletion attestations.

17. High child — health semantics hide blocked indexing

> **Class:** Child issue. **Parents:** #1 volume (writes blocked) and #2 colocation (liveness is process-local). **Likelihood:** Confirmed. **Blast radius:** operations and automated rollout decisions. **Evidence:** PROD-CONFIRMED API health 200 while indexing blocked; `main.py` deliberately keeps `/health` dependency-free and provides `/health/detail`. **Failure mode:** platform sees healthy process while job plane is capacity-blocked or dependencies are down. **Detection:** separate liveness, readiness, and worker-health/job-progress signals. **Short term:** alert on detailed readiness and queue progress, not `/health` alone. **Target:** role-specific health endpoints and deployment gates tied to readiness plus SLO burn.

18. High child — schema migration/startup coupling

> **Class:** Child issue. **Parent:** #2 colocation (same process boot is the migration engine). **Likelihood:** Medium. **Blast radius:** all non-health routes and every replica starting concurrently. **Evidence:** `main.py` runs `ensure_schema` with retries during deferred boot; `/health` bypasses readiness. **Failure mode:** DDL contention or failure leaves green liveness but 503 application; startup code becomes migration engine. **Detection:** migration version/duration/lock wait, boot readiness, schema failure count. **Short term:** expose readiness separately and ensure one migration owner. **Target:** explicit pre-deploy migrations with backward-compatible expand/contract rollout.

19. Medium-high bottleneck — external provider outage/fallback behavior

> **Class:** Bottleneck. **Saturates:** availability of OAuth, sync, embeddings, rerank, carousel, and YouTube. **Children:** none exclusive. **Likelihood:** Medium. **Blast radius:** OAuth, sync, embeddings, rerank, carousel, YouTube ingestion. **Evidence:** direct dependencies in config and clients; some local fallbacks exist, but no end-to-end outage matrix. **Failure mode:** cascading retries, partial artifacts, or silent quality downgrade. **Detection:** provider SLI, circuit state, fallback rate and result-quality marker. **Short term:** document stage behavior for 401/403/429/5xx/timeouts and fail fast by class. **Target:** dependency isolation, cached/degraded modes, provider abstraction, recovery queues.

20. Medium-high child — cost/egress/model-spend controls

> **Class:** Child issue. **Parents:** #6 LLM quota and #7 retry storms. **Likelihood:** High. **Blast radius:** budget and provider suspension. **Evidence:** high LLM concurrency, repeated image/frame work, remote previews; no supplied spend caps. **Failure mode:** retries/backfills or unauthenticated routes create runaway provider and egress cost; suspension removes LLM capacity. **Detection:** tokens/images/bytes and cost per workload/tenant/source. **Short term:** daily caps, anomaly alerts, backfill budgets, cache-hit reporting. **Target:** admission budget service, per-workload quotas, cost-aware routing and chargeback.

21. Medium-high child — observability gaps

> **Class:** Child issue. **Parents:** all primary bottlenecks (#1–#6, #8, #14, #15); this is the missing detection plane. **Likelihood:** High. **Blast radius:** delayed diagnosis across all incidents. **Evidence:** logging and detail health exist, but no supplied metrics backend/dashboards for queue depth, disk, or pool saturation. **Failure mode:** liveness remains green while capacity or dependencies block work. **Detection:** this issue is the missing detection plane itself. **Short term:** export disk, queue, pool, provider, stage, and consistency metrics with alerts. **Target:** service-level telemetry, traces with job/source IDs, dashboards and actionable alerts.

22. Medium-high child — capacity planning and SLO absence

> **Class:** Child issue. **Parents:** #1–#6 (meta). Without a model, every saturator recurs. **Likelihood:** High. **Blast radius:** every scaling and readiness decision. **Evidence:** configuration comments contain ad hoc throughput observations; no supplied formal SLO/capacity model. **Failure mode:** concurrency is tuned to peak rate instead of bottleneck headroom; incidents recur. **Detection:** demand, service time, utilization, queue age and headroom forecasts. **Short term:** baseline current rates and define initial SLOs below. **Target:** quarterly load tests and capacity model per resource/provider.

23. Medium-high bottleneck — single-volume/single-region durability

> **Class:** Bottleneck. **Saturates:** availability after volume or region loss, not live ingest rate. **Children:** #24, #25. **Likelihood:** Medium. **Blast radius:** non-reproducible uploads, YouTube copies, refs, and co-located backups. **Evidence:** configured paths in `backend/app/config.py`; retained source design in upload/YouTube routes; no supplied cross-region replica evidence. **Failure mode:** volume or region loss permanently removes app-owned sources and backups. Ranked below live-path saturators because it does not limit throughput until failure. **Detection:** replication status, restore-point age, object counts/checksums by category. **Short term:** inventory irreplaceable categories and copy to an independent backup destination. **Target:** multi-AZ object storage, versioning, lifecycle rules, and tested cross-region restore.

24. Medium-high child — backup/restore and DR uncertainty

> **Class:** Child issue. **Parent:** #23 single-volume/single-region durability (backups share the failure domain). **Likelihood:** Medium. **Blast radius:** complete metadata/vector/library recovery. **Evidence:** `backend/app/workers/backup.py` and `backup_dir` exist, but no supplied restore drill, RPO/RTO, or off-volume proof. **Failure mode:** backups are incomplete, co-fail with volume, or cannot recreate Postgres↔Qdrant consistency. **Detection:** last successful backup, offsite copy, checksum, quarterly restore result. **Short term:** document scope and perform isolated restore rehearsal. **Target:** defined RPO/RTO, automated PITR/snapshots, vector rebuild plan, and audited restore drills.

25. Medium-high child — upload/YouTube retention dependency

> **Class:** Child issue. **Parents:** #1 volume (same mount) and #23 durability (app is source of truth). **Likelihood:** Medium. **Blast radius:** irreplaceable source media and generated carousels. **Evidence:** `source="upload"` writes to volume; YouTube retained local path and cookies; 3.671 GiB confirmed. **Failure mode:** cleanup treats app-owned sources like reproducible Drive cache or upstream disappears. **Detection:** source category bytes, backup status, source availability. **Short term:** mark upload/YouTube as protected in cleanup code and manifests. **Target:** immutable source bucket with checksums, replication, explicit user retention policy.

26. Medium-high child — LLM cache poisoning/model/config mismatch

> **Class:** Child issue. **Parent:** #6 LLM quota (wasted tokens and duplicate generation). **Likelihood:** Medium. **Blast radius:** all reused themes/hooks/copy for a video. **Evidence:** LOCAL-ONLY provider:model cache ID and transcript hash; save fields include some versions but not every prompt/parameter/policy input. **Failure mode:** result generated under old prompt/config is served as current or malicious/corrupt payload persists. **Detection:** provenance completeness, schema validation, cache-hit quality sampling. **Short term:** include prompt/algorithm versions and validate payload shape. **Target:** canonical signed provenance hash, immutable cache entries, scoped invalidation.

27. Medium-high child — secrets/config/runtime-settings drift

> **Class:** Child issue. **Parents:** #2 colocation, #4 DB pool, and #6 LLM (replicas can over-admit). **Likelihood:** High. **Blast radius:** routing, costs, OAuth, concurrency, retention. **Evidence:** Pydantic env settings plus mutable singleton `AppSettings`, process cache, and local `.env` examples. **Failure mode:** replicas use different effective values; UI selection changes behavior without traceable release. **Detection:** sanitized effective-config hash per replica, settings revision, drift alert. **Short term:** publish non-secret effective configuration and revision; define precedence. **Target:** versioned config service/schema, audited changes, immutable release defaults, secret manager.

28. Medium-high bottleneck — OAuth refresh/token lifecycle

> **Class:** Bottleneck. **Saturates:** Drive-path availability (sync, preview, listing). **Children:** #32, #34. **Likelihood:** Medium. **Blast radius:** all Drive reads and Picker access. **Evidence:** plaintext token columns in `db/models.py`; refresh in multiple endpoints/client methods; no PKCE noted in `drive_oauth.py`. **Failure mode:** revoked/missing refresh token halts sync; concurrent refresh races; token exposure broadens access. Ranked below live compute/disk because it is an availability gate for Drive, not the first saturator under load. **Detection:** refresh failures by reason, token age, reconnect rate, audit access. **Short term:** centralize refresh with locking and redact logs; review encryption and revocation. **Target:** encrypted credential vault, per-tenant token records, PKCE/nonce-bound state, rotation and audit.

29. Medium child — `railway up` versus Git deployment drift

> **Class:** Child issue. **Parents:** #8 and #9 (can ship the buffering or pre-stream media path). **Likelihood:** Medium. **Blast radius:** every local-only safeguard and rollback. **Evidence:** Carousel README documents `railway up` and source redeploy; no supplied deployed SHA/digest evidence. **Failure mode:** production runs an unreviewed local tree or service versions diverge. **Detection:** `/version` with Git SHA/image digest; release inventory across services. **Short term:** stop untracked CLI deploys; record digest before any future change. **Target:** CI-built immutable artifacts, Git-protected promotion, signed provenance, one-click digest rollback.

30. Medium child — transcript grounding/timestamp correctness

> **Class:** Child issue. **Parent:** #15 consistency (timebase and artifact versions are not transactional). **Likelihood:** Medium. **Blast radius:** search results, frame picks, and factual carousel copy. **Evidence:** `VideoSegment`, Qdrant timestamp payloads, transcript guard, VTT/ASR/YouTube sources. **Failure mode:** offsets drift across containers/captions; correct text maps to wrong image; guard preserves wrong cue. Forces rework rather than saturating a resource first. **Detection:** sampled alignment error, cue/frame checks, timestamp monotonicity and duration bounds. **Short term:** store transcript source/timebase and reject invalid ranges. **Target:** normalized media timeline with provenance, confidence, automated alignment regression set.

31. Medium child — public `/test` and admin-like surface/auth boundaries

> **Class:** Child issue. **Parents:** #6 LLM and #20 spend (unbounded admission of costly work if reachable). **Likelihood:** Medium, exposure unverified. **Blast radius:** settings, indexing, uploads, generation cost, data access. **Evidence:** CODE-CONFIRMED `/test` pages and broad backend routers; no global auth middleware visible in `main.py`. This report does **not** claim public Internet reachability. **Failure mode:** if externally reachable, unauthenticated users may invoke costly or mutating operations. **Detection:** route inventory with ingress/auth policy; anonymous integration tests; access logs. **Short term:** confirm Railway ingress and gate `/test`, settings, retries, deletes, uploads, and index controls. **Target:** centralized authentication/RBAC, CSRF protection, environment-specific route registration.

32. Medium child — shared Drive account / tenant isolation

> **Class:** Child issue. **Parents:** #14 Drive listing and #28 OAuth (one `DriveUser` is one quota and token). **Likelihood:** High if multiple users are exposed. **Blast radius:** cross-user folder, token, and library access. **Evidence:** repeated `select(DriveUser).limit(1)` and no request user/tenant foreign key in core entities. **Failure mode:** any user sees or mutates the globally connected account and permanent library. Performance effect is a single Drive identity/quota, not a separate saturator. **Detection:** account identity per request, cross-user access tests, route authorization logs. **Short term:** treat deployment as single-tenant and restrict network/user access explicitly. **Target:** authenticated tenant/user IDs on all resources, row-level authorization, tenant-scoped vectors/objects.

33. Medium child — data privacy/PII/face biometric governance

> **Class:** Child issue. **Parent:** none — governance, not a live capacity constraint. **Likelihood:** High as a governance concern. **Blast radius:** people represented in media and operators. **Evidence:** emails/tokens, face embeddings, body signatures, person labels, thumbnails, reverse-web match records in `db/models.py`. **Failure mode:** over-retention, unauthorized identification, difficult deletion/export, provider disclosure. Does not saturate disk/CPU/DB/LLM/queues by itself. **Detection:** data inventory, access audit, retention/deletion verification, provider data-flow register. **Short term:** document purpose, access, retention, and deletion; disable optional identification features unless approved. **Target:** privacy-by-design controls, consent/purpose boundaries, encryption, scoped roles, deletion workflow. This is not a legal conclusion.

34. Medium child — Google Picker referrer/config coupling

> **Class:** Child issue. **Parents:** #28 OAuth and #27 config drift. **Likelihood:** Medium. **Blast radius:** folder onboarding. **Evidence:** `/api/drive-token` returns browser key/app ID and explicitly depends on Drive/Picker APIs and referrer restrictions. **Failure mode:** domain/config change breaks Picker while OAuth remains connected. Onboarding availability, not ingest throughput. **Detection:** Picker launch/select success by origin, key restriction errors. **Short term:** preflight configured origin/app ID and document allowed domains. **Target:** environment-specific OAuth/Picker config tested during release.

35. Medium child — monolithic backend and Carousel files

> **Class:** Child issue. **Parent:** #22 capacity/SLO model (change risk on the performance path). **Likelihood:** High for change defects. **Blast radius:** carousel and worker maintainability. **Evidence:** `routers/carousel_script.py` is thousands of lines; `workers/indexer.py` and Carousel pages combine many stages. **Failure mode:** hidden coupling, slow review, broad regression surface. **Detection:** file churn, defect concentration, cycle dependencies, test duration. **Short term:** freeze new responsibilities and extract pure stage contracts. **Target:** bounded modules/services for ingestion, retrieval, generation, and media delivery.

36. Medium child — duplicated main versus `/test` UI logic

> **Class:** Child issue. **Parent:** #31 `/test` surface. **Likelihood:** High. **Blast radius:** inconsistent behavior and security controls. **Evidence:** main Carousel page plus multiple `/test` pages/components and separate API helper modules. **Failure mode:** fixes land in one UI path; test surface becomes de facto production with different defaults. **Detection:** route/component duplication map and parity tests. **Short term:** label/gate `/test` and share core hooks/components. **Target:** one domain package and feature flags, with test routes as thin harnesses.

37. Medium child — weak frontend test coverage

> **Class:** Child issue. **Parents:** #8 and #9 (range/proxy regressions escape). **Likelihood:** Medium-high. **Blast radius:** proxy, OAuth, picker, studio state, and playback. **Evidence:** repository tests are backend-heavy; no supplied comprehensive browser suite for current and `/test` flows. **Failure mode:** deployment passes build but breaks range headers, state transitions, or origin config. **Detection:** coverage and critical-journey pass rate. **Short term:** add smoke tests for OAuth return, folder select, search, range seek, and carousel run. **Target:** contract + browser tests in CI against immutable preview environment.

38. Medium child — search relevance regression detection

> **Class:** Child issue. **Parents:** #15 consistency, #6 LLM rerank, and #22 missing quality SLOs. **Likelihood:** High as models/thresholds change. **Blast radius:** user trust and carousel grounding. **Evidence:** many configurable thresholds, fusion weights, rerank toggles, model choices, and source branches. **Failure mode:** latency improves while recall/precision silently declines; one modality dominates. Not a saturating resource. **Detection:** labeled query set, nDCG/Recall@K, grounded-moment accuracy, no-result and reformulation rates. **Short term:** capture a representative golden set and pin model/config provenance. **Target:** offline eval gates plus online quality monitoring and rollback.

6. Failure containment and recovery design

```
signal: disk/auth/quota/provider/DB/vector failure
  -> classify deterministic vs transient
  -> close admission circuit for affected resource class
  -> persist job lease/error/next-attempt; do not erase source relation
  -> preserve or quarantine partial artifacts with owner metadata
  -> continue unaffected API/search paths in degraded mode
  -> reconcile Postgres + Qdrant + media manifest
  -> operator approves bounded replay
  -> canary, observe, then reopen circuit
```

Containment boundaries should be resource-specific: Drive outage must not disable search over retained assets; Gemini quota must not block local previews; Qdrant outage must not corrupt Postgres; disk pressure must reject new durable writes before it prevents reads; Carousel provider failure must leave a persisted resumable job.

7. Roadmap and operating model

Immediate containment (0–48 hours)

- **Ship auto-clean of leftover Drive downloads** in the indexer success and ERROR paths. This is the required control for bottleneck #1. Manual cleanup already held (77.107 GiB free; 21 keep-set files) but will not prevent the next fill.
- Keep `reindex_errored_files` and `reindex_skipped_files` disabled until auto-unlink ships. Auto-index may stay on for PENDING-only work.
- Freeze operator deletions outside a signed manifest; protect upload, YouTube, unknown/orphan, active PROCESSING, skipped-with-Media, and references.
- Capture Postgres backup, Qdrant inventory, volume file manifest, deployed service SHAs/digests, and effective config.
- Add disk bytes/inodes, leftover Drive-cache count after terminal status, queue age, DB pool, provider 429, and index-progress alerts.
- Verify the keep set remains YouTube 7 / SKIPPED 11 / PROCESSING 1 / orphan 2; do not reclean those classes.

Acceptance: indexer unlinks Drive leftovers on PROCESSED+Media and ERROR/no-Media; 24 hours without unplanned cache growth; 100% of current files classified; latest backup independently restorable at least to inventory level.

One week

- Production-canary end-to-end Range for cached and uncached Drive video, including Next proxy, seek, cancel, 206/416, and RSS.
- Fix/stream the non-YouTube download path or set an explicit size limit.
- Create role-specific readiness/worker health and a release `/version`.
- Add time-based status flush and durable claim/attempt fields.
- Restrict/gate `/test` and admin-like mutating routes based on verified ingress.

Acceptance: 1 MiB range transfers ≤1.1 MiB per app hop; API p95 unaffected by a controlled index run; no anonymous mutation in external ingress test.

30 days

- Split API and one-replica indexer using immutable same-image digest and explicit connection budgets.
- Introduce transactional outbox and durable job queue for embeddings/index/generation.
- Reconcile Postgres↔Qdrant↔volume daily and version all vector artifacts.
- Move irreplaceable uploads/YouTube/refs and backups to independent object storage.
- Establish labeled search and transcript/frame alignment evaluation sets.

Acceptance: indexer restart does not restart API; zero lost acknowledged jobs in restart test; 100% PROCESSED rows have current artifact manifest or visible exception; restore drill meets provisional RPO/RTO.

90 days

- Resource-class worker pools and autoscaling; provider gateway with priorities and budgets.
- Incremental Drive Changes-based inventory and tenant-scoped auth/data model if multi-user access is intended.
- Expand/contract migration pipeline, CI browser/contract/load tests, and signed deployment provenance.
- Cross-region recovery, quarterly game days, privacy/retention control implementation.

Acceptance: all suggested SLOs measured for 30 days; capacity forecast maintains ≥30% headroom; tested region-loss and provider-outage runbooks; relevance release gate active.

8. Suggested SLOs and measurable gates

- **API availability:** 99.9% monthly for authenticated non-generation endpoints.
- **Search:** p95 <750 ms excluding explicit LLM stages; <1% 5xx over 15 minutes.
- **Carousel jobs:** 99% accepted jobs reach terminal state; p95 queue wait <30 s; no duplicate billable run for same idempotency key.
- **Index freshness:** 95% eligible new files searchable within 15 minutes under normal provider health.
- **Queue:** oldest ready job <5 minutes; zero unbounded retries; DLQ age <24 hours.
- **Storage:** alert 70%, stop nonessential writes 80%, hard gate 85%; preserve max(10 GiB, 15%) free.
- **Streaming:** valid single-range 206; ≤10% byte overhead; seek p95 <2 s cached/<4 s Drive.
- **Consistency:** ≥99.9% current artifact completeness; all exceptions identified and replayable.
- **Recovery:** provisional RPO 24 h/RTO 4 h until business owners set stricter targets.
- **Quality:** no release with >5% relative drop in Recall@10/nDCG or grounded timestamp accuracy.

9. Production readiness checklist

- ☐ Immutable image digest and Git SHA visible for every service.
- ☐ Explicit liveness, readiness, and worker progress checks.
- ☐ API/indexer connection and CPU/memory budgets load-tested.
- ☐ Durable queue, leases, fencing, idempotency, retry budget, and DLQ.
- ☐ Range and proxy conformance tests pass on deployed canary.
- ☐ Storage categories, quotas, retention, and deletion manifests active.
- ☐ Upload/YouTube/reference sources replicated independently.
- ☐ Postgres/Qdrant backup and restore drill completed.
- ☐ Schema migration separated from normal replica startup.
- ☐ OAuth state/PKCE/token encryption and tenant model reviewed.
- ☐ `/test`, settings, index, retry, delete, upload, and feedback routes authorized.
- ☐ Provider quotas, fallback, spend limits, and data handling documented.
- ☐ Queue/disk/pool/provider/quality dashboards and alerts owned.
- ☐ Search and carousel grounding regression suite gates release.
- ☐ PII/face/body data access, retention, export, and deletion controls approved.

10. Incident response and rollback checklist

1. Identify failure class and freeze only affected admission/retry circuits.
2. Record service digest/SHA, config revision, DB migration version, queue state, disk manifest, and provider status.
3. Preserve logs and source artifacts; never “fix” capacity by deleting unknown files.
4. Snapshot authoritative Postgres before state repair; snapshot/record Qdrant collection aliases.
5. Roll back to a known image digest only if schema is backward-compatible; otherwise execute the documented database rollback/forward-fix.
6. For ENOSPC: stop writers, reserve headroom, classify partials, reconcile, then delete only manifest-approved batches.
7. For Qdrant inconsistency: remove the collection/alias from serving if necessary, replay outbox/rebuild, then compare counts/sample quality.
8. For OAuth/provider failure: close that dependency circuit, retain jobs with next-attempt, and keep local search/preview available.
9. Canary one job/file/query before reopening; monitor error rate, queue age, disk slope, cost, and quality.
10. Publish timeline, impact, cause, permanent actions, and validation evidence.

11. Source-based cache deletion matrix

Source/category	Default	Live on disk (07:16 UTC)	Preconditions	Rollback evidence
upload source	KEEP	0	explicit user/admin retention action + independent backup	checksum and restore copy
YouTube retained	KEEP	7 files / 3.671 GiB	explicit policy; dependent derivatives verified	retained source checksum
Drive PROCESSED/Media	**AUTO-DELETE after Media**	0 (182 previously removed)	Media committed; Drive readable; Range available; no active lease	manifest + source ID/ETag
Drive ERROR/no-Media	**AUTO-DELETE after ERROR**	0 (1,023 / 41.391 GiB previously removed; 4,797 ERROR rows remain in DB)	no Media; retries re-download from Drive	manifest + dry-run reconciliation
Drive SKIPPED	KEEP until classified	11 files	reason reviewed and downstream use absent	manifest
Drive PROCESSING	KEEP	1 file (proof `11bY89L94ctjD…`, 24.78 MiB)	wait for terminal status, then apply success/ERROR rule	claim/lease record
duplicate case variants	verify only	0 (127 previously removed)	content/source identity and referenced canonical path	pre/post cleanup manifest
unknown/orphan	KEEP/quarantine	2 files	classified by source/hash/ownership	quarantine record
`.partial`/temp	AUTO-DELETE after failed/abandoned job	0	no live lease; age exceeds max job+retry duration	owner/age audit
frames/thumbnails	derived lifecycle	keep with parent	parent/current version exists or rebuild accepted	parent/version manifest
carousel reference uploads	KEEP	n/a	explicit reference deletion and save dependency check	save/reference audit

The prior **36.433 GiB processed-Drive** and **41.415 GiB ERROR/no-Media** envelopes were operator-reclaimed (182 and 1,023 files). They must not be added back to the current **4.617 GiB** keep set. The original **83.135 GiB / 1,353 files** figure is the incident fill, not the current inventory. Auto-clean in the indexer is now the required control so those leftover classes cannot refill the volume.

12. Evidence index and limitations

Primary code evidence:

- `backend/app/main.py`, `config.py`, `runtime_settings.py`
- `backend/app/db/models.py`, `schema.py`, `advisory_locks.py`, `deadlock.py`
- `backend/app/drive/google_client.py`, `media_cache.py`
- `backend/app/routers/drive_oauth.py`, `drive.py`, `index.py`, `search.py`, `settings.py`, `youtube.py`, `carousel_script.py`
- `backend/app/pipelines/common.py`, `image.py`, `video.py`
- `backend/app/workers/indexer.py`, `auto_indexer.py`, `maintenance.py`, `embed_queue.py`, `index_batch.py`
- `backend/app/qdrant/*`, especially `video_transcripts.py`
- `backend/app/search/moments.py`, `carousel_pipeline.py`, `carousel_frame_select.py`
- `backend/app/llm/carousel_llm.py`, `openrouter.py`
- `carousel-frontend/lib/api-proxy.ts`, `browser-frame-capture.ts`, `app/carousel/page.tsx`, `app/test/*`
- `docs/indexer-service.md`, Railway JSON files, `.env` examples

Limitations:

- This refresh queried production **read-only**: public `/health`, `/health/detail`, `/index`, plus Railway SSH SELECT/stat of disk, `/app/data/videos`, `app_settings`, `drive_files`/`media` counts, and the proof video. No settings write, cache delete, deploy, or DB mutation.
- Deployed image digests, backup destination, ingress policy, and region/plan metadata were not independently re-verified in this pass.
- Security/privacy items are architecture and governance risks, not claims of exploitation or legal noncompliance.
- Auto-clean of leftover Drive downloads is a required control described here; it is **not** implemented in the current indexer success/ERROR paths.

13. Reproducible rendering

From repository root:

```
node scripts/render-architecture-report.mjs \
  docs/full-system-architecture-and-risks.html \
  docs/artifacts/full-system-architecture-and-risks.pdf
```

The renderer blocks external requests, uses installed local Chromium through Playwright, prints deterministic A4 output, and retains the legacy no-argument command for the existing bottleneck report.